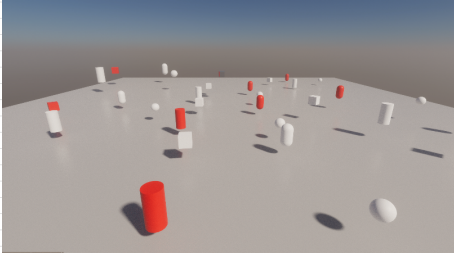


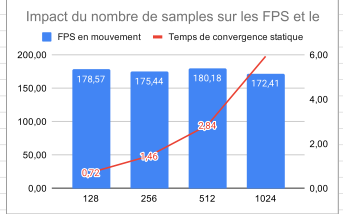
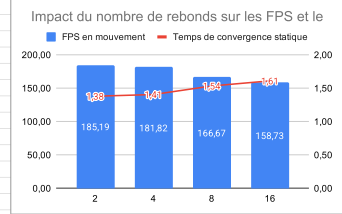
Test de perf du Path Tracing : Formes simples (32 assets 3D d'Unity)				
Max Depth (Rebonds)	Max Samples (Echantillons)	GPU (ms)	FPS en mouvement	Temps de convergence statique
2	256	5,4	185,19	1,38
4	256	5,5	181,82	1,41
8	256	6	166,67	1,54
16	256	6,3	158,73	1,61
4	128	5,6	178,57	0,72
4	256	5,7	175,44	1,46
4	512	5,55	180,18	2,84
4	1024	5,8	172,41	5,94



Résolution	1920x1080
Denoyer NVIDIA	Désactivé
Nb triangles	25,8k
Nb points	17,9k
Batches	68
Setpass calls	6

Le moteur gère bien l'impact des rebonds sur une scène simple
tourne généralement à + de 150 FPS (- de 7ms par image, très bonne perf)
doubler les échantillons double clairement le temps de convergence

Passer de 2 à 16 rebonds ne fait passer le GPU que de 5,4 ms à 6,3 ms
logique car formes simples



dans un moteur physique, un matériau presque blanc pur renvoie presque 100% de l'énergie lumineuse à chaque rebond. Avec le matériau rouge, ça absorrait une grande partie de la lumière (le vert et le bleu du spectre).
avec la maquette presque blanche, les rayons ne perdent presque pas d'énergie. L'algo est donc forcé de calculer tous les rebonds pour chaque rayon avant de les arrêter, chacun rebondit partout dans la pièce sans s'épuiser, ce qui sature la carte graphique.

Test de perf du Path Tracing : Maquette de musée, texture unie (SketchUp)				
Max Depth (Rebonds)	Max Samples (Echantillons)	GPU (ms)	FPS en mouvement	Temps de convergence statique
2	256	8	125,00	2,05
4	256	22,5	44,44	5,76
8	256	27,5	36,36	7,04
10	256	25,5	39,22	6,53
16	256	20	50,00	5,12
4	128	22,25	44,94	2,85
4	256	22	45,45	5,63
4	512	23	43,48	11,78
4	1024	25,5	39,22	26,11
16	256	70	14,29	17,92
6	256	24,5	40,82	6,27
4	128	24,5	40,82	3,14

Convertir STL pour Unity
https://github.com/kari-jpb_Slt

Résolution	1920x1080
Denoyer NVIDIA	Activé
Nb triangles	3,4k
Nb points	10,2k
Batches	7
Setpass calls	7

test avec matériau quasi blanc (qui absorbe moins la lumière)

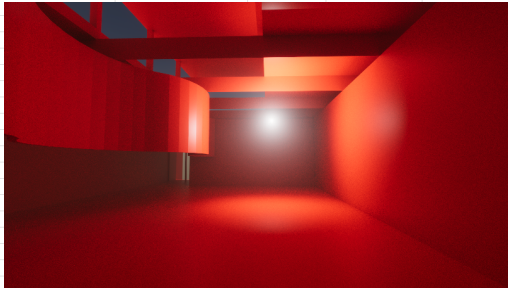
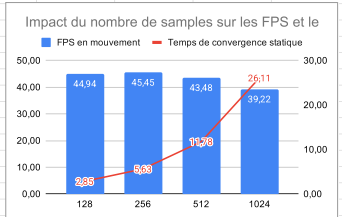
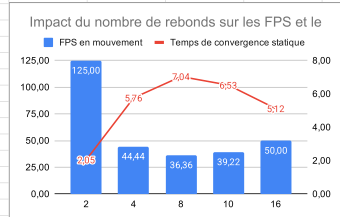
test avec matériau noir (pour voir impact de l'absorbo)

opti statistique du path tracing

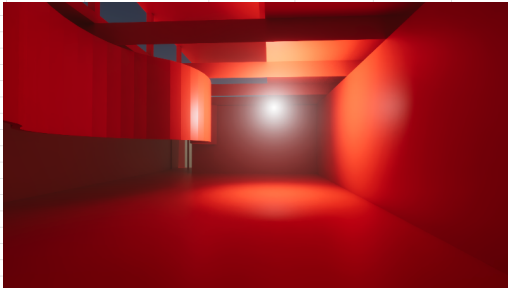
si, au bout de 4 ou 5 rebonds, le rayon a perdu presque toute son énergie (par exemple parce qu'il a rebondi sur des murs rouges foncés qui absorbent la lumière)
l'algo tue le rayon prématurément plutôt que de calculer les 16 rebonds pour rien --> très peu de rayons atteignent réellement 16 rebonds
--> très peu de rayons atteignent réellement 16 rebonds

On voit qu'on est passés de ~180 FPS (formes simples en extérieur) à ~45 FPS dans la maquette
logique car la lumière reste coincée dans la pièce et rebondit partout, ce qui donne beaucoup de travail au Path Tracer.

dans un moteur physique, un matériau presque blanc pur renvoie presque 100% de l'énergie lumineuse à chaque rebond
avec le matériau rouge, ça absorrait une grande partie de la lumière (vert et bleu du spectre)
avec la maquette presque blanche, les rayons ne perdent presque pas d'énergie
l'algo est donc forcé de calculer tous les rebonds pour chaque rayon avant de les arrêter, chacun rebondit partout dans la pièce sans s'épuiser, ce qui peut saturer le GPU



Sans NVIDIA Optix Denoyer



Avec NVIDIA Optix Denoyer

permet de garder un nombre d'échantillons bas (ex: 256) tout en ayant une image propre

Test de perf du Path Tracing : Maquette de musée, texture claire, 3 lumières IES + soleil				
Max Depth (Rebonds)	Max Samples (Echantillons)	GPU (ms)	FPS en mouvement	Temps de convergence statique
2	256	9,65	103,63	2,47
4	256	26	38,46	6,66
6	256	27,5	36,36	7,04
12	256	21,75	45,98	5,57
6	64	27,75	36,04	1,78
6	128	27,5	36,36	3,52
6	512	27,75	36,04	14,21
6	1024	31,5	31,75	32,26
12	1024	25,5	39,22	26,11

Librairie IES
<https://ieslibray.com/>

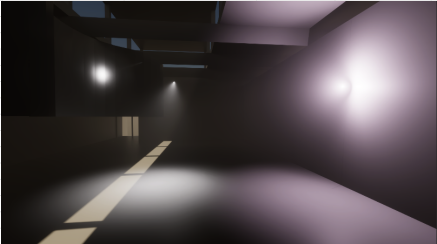
Résolution	1920x1080
Denoyer NVIDIA	Activé
Nb triangles	3,4k
Nb points	10,2k
Batches	6
Setpass calls	6

cas extrême

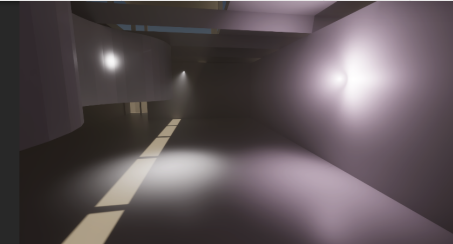
à Depth 2, les zones qui ne sont pas frappées directement par la lumière sont plongées dans le noir
à Depth 12, la lumière rebondit sur le sol clair et vient éclairer le plafond de manière douce

que je mette 64, 128, 256 ou 512 samples, le GPU met toujours ~27,5 ms et on reste à ~36 FPS
quand je bouge la cam', le moteur calcule un minimum de rayons par frame (que j'ai setup ici à 1)
le max samples agit sur le nombre de calculs à faire uniquement quand l'image est statique
--> d'où le temps de convergence qui augmente "proportionnellement"

Max Depth : 2, Max Samples : 256



Max Depth : 12, Max Samples : 1024



depth 12 plus rapide que depth 6
probablement une optimisation interne de unity
qui doit tuer les rayons de lumière qui n'ont plus assez d'énergies

bon compromis : Max Depth 6 et Max Samples 256
autour de 30-40 FPS et entre 7-14 sec d'attente pour avoir une image fixe nette

Test de perf du Path Tracing : Maquette de musée, texture claire et texture de verre sur les murs, soleil venant de la droite				
Max Depth (Rebonds)	Max Samples (Echantillons)	GPU (ms)	FPS en mouvement	Temps de convergence statique
2	256	9.05	110.50	2.32
4	256	22.5	44.44	5.76
6	256	28.5	35.09	7.30
12	256	28.2	35.46	7.22
6	64	29	34.48	1.86
6	128	28.75	34.78	3.68
6	512	29	34.48	14.85
6	1024	32	31.25	32.77
12	1024	28.25	35.40	28.93

Librairie IES
<https://testlibrary.com/>

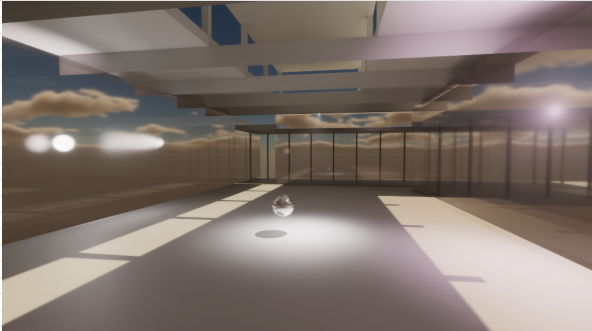
Résolution	1920x1080
Denoyer NVIDIA	Activé
Nb triangles	2.4k
Nb points	4.4k
Batches	6
Setpass calls	7

cas extrême

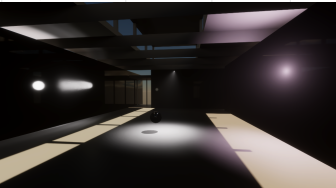
Les perfs tiennent le coup ! pas beaucoup de diff par rapport aux tests précédents (~1 ms de diff)
on a toujours ici le cas où unity arrête de calculer les rayons quand ils ont plus assez d'énergie (cf cas extrême)
-> ça a un nom : algo de la roulette russe : <https://computergraphics.stackexchange.com/questions/2316/is-russian-roulette-really-the-answer>

à chaque calcul sur la rayon, soit on le tue, soit on augmente artificiellement sa lumière pour compenser les pertes
algo non-biaisée / impartial

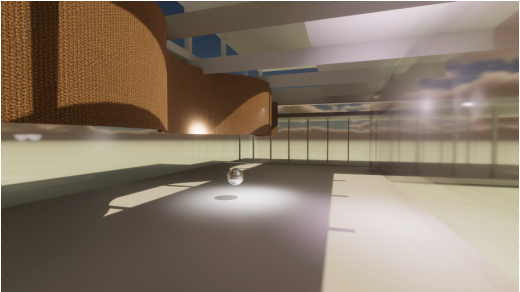
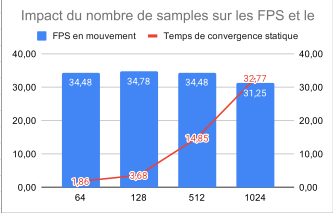
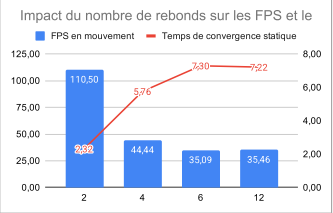
https://pbr-book.org/3ed-2018/Monte_Carlo_Integration/Russian_Roulette_and_Splitting



max Depth 2



logique : le rayon meurt dans le mur avant de pouvoir en sortir
confirme qu'il faut au moins 4-6 rebonds pour pouvoir laisser entrer la lumière



ajout du tissu et optimisation des normales pour un rendu plus lisse